PROYECTO FINAL

Sistema progresivo de análisis de sentimiento y generación de resúmenes

Deep Learning aplicado al dataset Amazon Fine Food Reviews

Narrativa del proyecto

- Clasificación multiclase de sentimiento: negative / neutral / positive.
- Generación automática de resúmenes cortos a partir de reseñas largas.
- Construcción por etapas para comparar el aporte real de cada arquitectura.

Ruta de exposición

01	02	03	04	05
Introducción y EDA	MLP (base)	1D-CNN	RoBERTa + Generativos	Fine-tuning y Gradio

Introducción, problema y objetivos

Problema que resolvemos

- Amazon concentra cientos de miles de reseñas; leerlas manualmente no escala.
- Clasificar sentimiento ayuda a ordenar, filtrar y agregar señales para decisiones de negocio.
- Resumir reseñas largas permite mostrarlas en dashboards, listados o búsquedas sin perder la idea central.

Objetivos del proyecto

- Definir una línea base con MLP.
- Comparar una arquitectura profunda especializada (1D-CNN).
- Evaluar transferencia con RoBERTa zero-shot.
- Agregar un componente generativo con T5 y luego adaptarlo con fine-tuning parcial.

Por qué Deep Learning aquí

568k+
reseñas reales

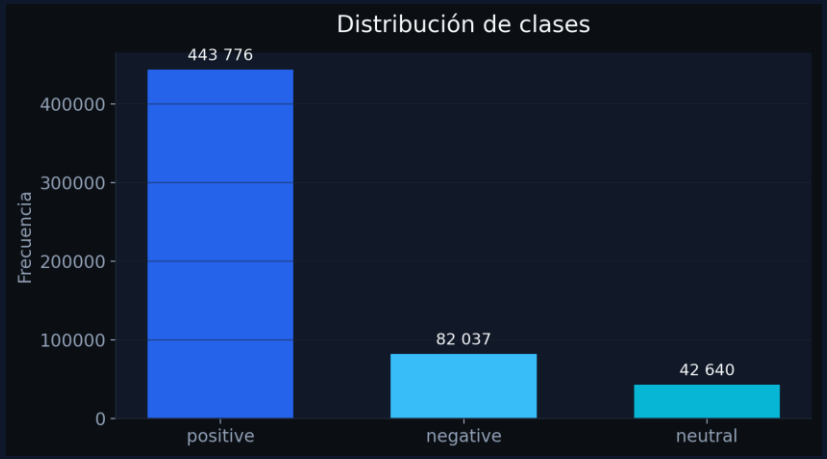
3
clases de sentimiento

5
etapas progresivas

- Texto natural con negación, contexto y variación de estilo: difícil para modelos lineales puros.
- El curso pedía ampliaciones progresivas; este problema encaja perfecto para ir de modelos base a preentrenados y generativos.
- La comparación entre arquitecturas permite defender decisiones, no solo mostrar métricas.

Dataset y EDA

Distribución de clases



568 454
reseñas totales

78.07%
clase positive

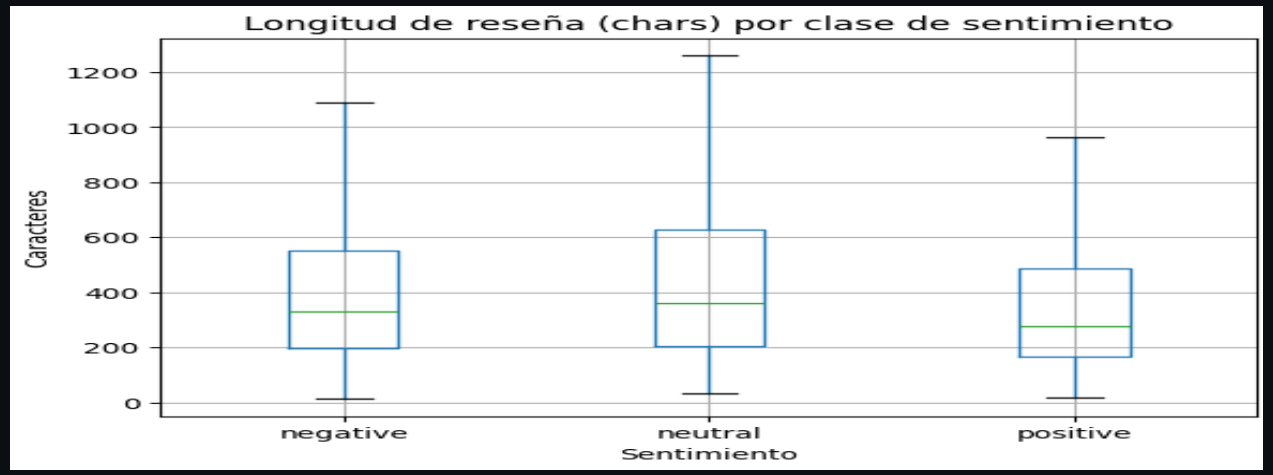
417
media de caracteres

290
mediana

20 517
máximo

70/15/15
split estratificado

Idea clave: el desbalance obliga a mirar F1 macro y Recall macro;



La función `limpiar_texto` elimina todo lo que no sea alfabético. Esto simplifica el vocabulariopara TF-IDF/embeddings, pero tiene costo: se pierden números (p. ej. "5/5"), puntuación enfática(p. ej. "!!!") y negaciones con apóstrofe ("don't" -> "don t"). Las etapas que usan Transformers(RoBERTa, T5) trabajan sobre el texto ya tokenizado por su propio tokenizer, por lo que la pérdidaen esas etapas es menor.

MLP como línea base

Por qué empezar con MLP

- Entrega una referencia clara de desempeño y costo computacional.
- TF-IDF + bigramas captura vocabulario discriminativo sin requerir GPU pesada.
- Sirve para demostrar qué problemas siguen abiertos: orden, negación, sarcasmo y contexto.

Configuración usada

- TfidfVectorizer: 10 000 features, unigramas + bigramas.
- MLPClassifier: 1 capa oculta de 48 unidades, ReLU, Adam, L2.
- Undersampling de la clase positive a 120 000 muestras.
- Early stopping manual con warm_start.

Arquitectura del baseline

Texto limpio → **TF-IDF 10k** → **MLP 48 hidden** **3 clases**

Limitación estructural a defender

Bag-of-words no modela el orden de las palabras. Por eso frases como “not good” o reseñas con giro de sentido pueden confundir al modelo.

Resultados del MLP

Buen desempeño global y balance razonable entre clases, pero con fallas previsibles por falta de contexto.

0.8804
Accuracy

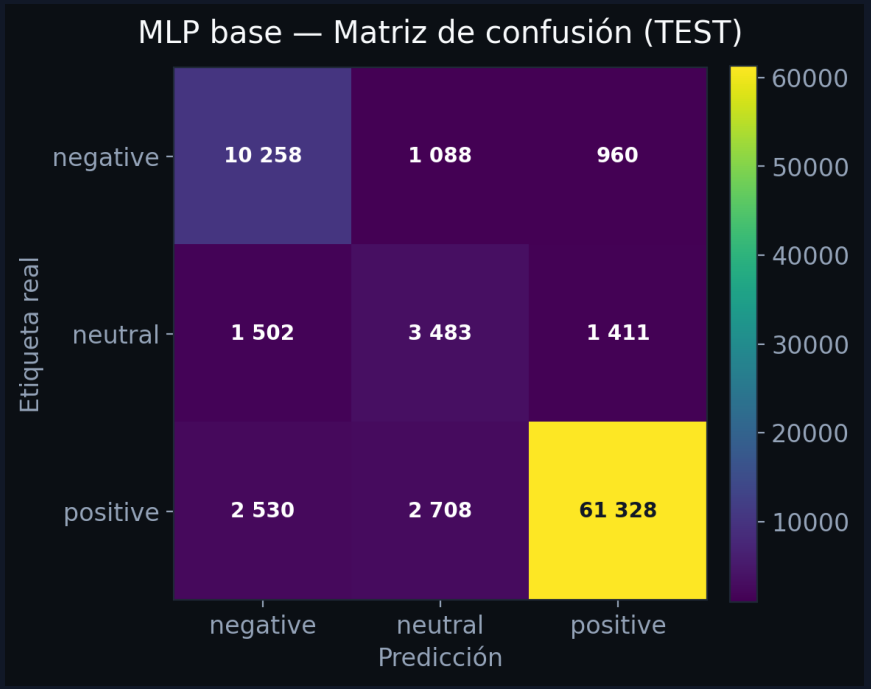
0.7408
F1 macro

0.8846
F1 weighted

Patrones de error más importantes

- Negación contextual: “was skeptical… turned out great”.
- Sarcasmo y comparaciones: combina tokens negativos con sentido final positivo.
- Reseñas muy cortas: un solo adjetivo domina toda la decisión.

Matriz de confusión (TEST)



1D-CNN: por qué se eligió y cómo está armada

Se priorizó captura de patrones locales y entrenamiento paralelo frente a alternativas recurrentes.

Justificación de la arquitectura

- Captura n-gramas locales muy relevantes para sentimiento: “not good”, “highly recommend”.
- Entrena en paralelo y suele ser más estable que LSTM/GRU sobre grandes volúmenes.
- Logra más capacidad expresiva que TF-IDF sin el costo de un Transformer entrenado.

Entrenamiento

- Adam lr=1e-3, batch size 64.
- Hasta 12 épocas con EarlyStopping y ReduceLROnPlateau.
- max_len=100 para equilibrar cobertura y costo.

Flujo de la 1D-CNN

Embedding	Conv1D	Global Max Pooling	Dense 48	Softmax
10k → 64	96 filtros kernel=5	reduce tiempo	ReLU + L2	3 clases

Idea clave: la CNN ya incorpora orden local de palabras, algo que el MLP con TF-IDF no puede representar directamente.

RoBERTa zero-shot: valor y limitación real

Qué se usó

- Modelo: cardiffnlp/twitter-roberta-base-sentiment (~125M parámetros).
- Alineado a 3 clases de sentimiento.
- Uso en inferencia directa: sin fine-tuning adicional.

Métricas obtenidas

0.8344
Accuracy

0.6166
F1 macro

0.6182
Recall macro

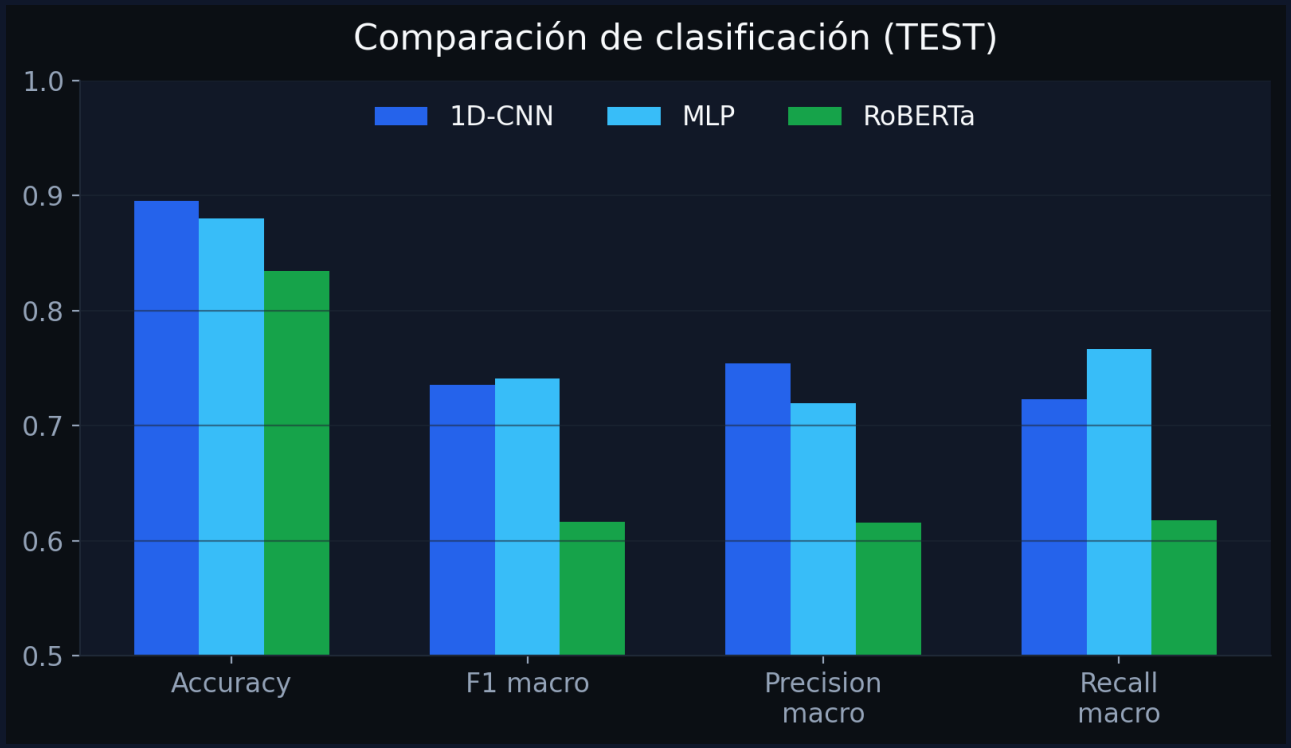
La explicación importante

- El modelo fue preentrenado en tweets, no en reseñas de comida.
- Esea diferencia de entrenamient baja el rendimiento al aplicarlo tal cual.
- Conclusión : zero-shot no bastó; un fine-tuning de sentimiento probablemente cambiaría mucho el resultado.

Eresultado NO invalida Transformers. Más bien valida que la adaptación al dominio importa.

Resultados CNN y comparación contra MLP / RoBERTa

La CNN gana en accuracy y precision macro, pero el MLP sigue fuerte en balance entre clases.



Lectura que conviene decir

- La CNN logra la mejor accuracy: 0.8954.
- También mejora precision macro: 0.7540.
- El MLP conserva mejor recall macro y F1 macro.
- Conclusión : no hay ganador absoluto; depende del criterio de optimización.

0.8954
Accuracy CNN

0.7540
Precision macro

Componente generativo con T5-small zero-shot

El sistema no solo clasifica: también intenta comprimir reseñas largas en títulos cortos.

Por qué T5-small

- Arquitectura encoder-decoder text-to-text nativa para summarization.
- Tamaño manejable en GPU gratuita de Kaggle (~60M parámetros).
- Se evaluó primero zero-shot para tener una línea base antes de adaptar al dominio.

Cómo se evaluó

- Prompt: “summarize: <review>”.
- Beam search con num_beams=4.
- ROUGE-1/2/L sobre 500 reseñas del test set.

Por qué el ROUGE sale bajo

- Los Summary reales del dataset son ultracortos: etiquetas de 3 a 5 palabras.
- T5 zero-shot tiende a generar frases más largas o parafraseadas.
- Eso castiga el solapamiento léxico aunque el contenido tenga sentido.

Ejemplo cualitativo

```
Ejemplo 2  [sentiment: positive]

REVIEW (primeras 500 chars):
i purchased this coffee at a local store a few months back and loved it it is by far the best
coffee i have ever had and i m not a crazy coffee drinker like some i got this on auto deliver
y and saved even more yum i m already for a cup now

SUMMARY REAL (ground truth):
best coffee ever

SUMMARY GENERADO (T5-small zero-shot):
i m not a crazy coffee drinker like some i got this on auto delivery and saved even more yum i
m
```

Fine-tuning parcial de T5-small

La mayor mejora del proyecto aparece cuando se adapta el generador al estilo real del dataset.

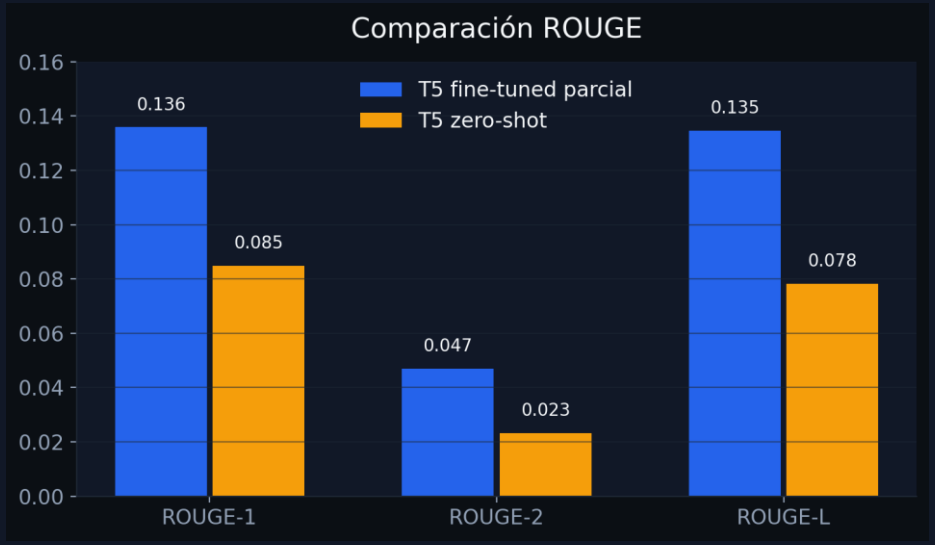
Qué se entrenó y qué no

- Encoder completo congelado.
- Primeros 4 bloques del decoder congelados.
- Últimos 2 bloques + final_layer_norm + lm_head entrenables.
- ≈ 41 % de parámetros entrenables (~25M de ~60M).

Frozen | Frozen | Frozen | Trainable | Trainable

Decoder blocks

1 epoch • batch 8 | fp16 | lr 2e-4



+60% ROUGE-1

+102% ROUGE-2

+72% ROUGE-L